

LAPORAN PROJECT

Centralized Smart Lighting Control System Based on IoT



NAMA ANGGOTA KELOMPOK :

M. Gyan Hendriansyah	20230140204
Aldys Igidia Triatmaja	20230140207
Syafrina Metavianida	20230140211
Asti Nurul Utami	20230140217
Nayla Salwa Hayati	20230140218
Muhammad Islam	20230140220
Muhammad Haris	20230140235
Nabiilah 'Afiifah Zalfa' S	20230140243
Choirul Amir Siregar	20230140249

PROGRAM STUDI TEKNOLOGI INFORMASI

FAKULTAS TEKNIK

UNIVERSITAS MUHAMMADIYAH YOGYAKARTA

2025

Daftar Isi

BAB I – PENDAHULUAN.....	2
Latar Belakang.....	3
Rumusan Masalah	3
Tujuan Proyek.....	3
Manfaat Proyek	3
BAB II – SPESIFIKASI TEKNIS	3
Perangkat Lunak (Software Stack).....	4
1. Frontend (Antarmuka Pengguna).....	4
2. Backend (Server & API).....	4
3. Database	4
Perangkat Keras (Hardware & IoT Layer).....	4
1. Mikrokontroler	5
2. Sensor Cahaya (LDR – Light Dependent Resistor).....	5
3. Perangkat Pencahayaan (Aktuator)	5
Protokol Komunikasi dan Keamanan	5
1. Protokol Komunikasi	5
2. Keamanan Sistem.....	6
BAB III– PERANCANGAN SISTEM (UML).....	7
3.1 Arsitektur Sistem.....	7
3.2 Use Case Diagram.....	7
3.3 Alur Kerja (Activity Diagram)	8
BAB IV– STRUKTUR DATA DAN PERANCANGAN DATABASE (ERD)	9

4.1 Entitas Utama dan Skema Tabel	10
4.2 Hubungan Antar Entitas (Relasi)	11
BAB V– KESIMPULAN.....	12

BAB I – PENDAHULUAN

Latar Belakang

Perkembangan *Internet of Things* (IoT) memungkinkan pengendalian perangkat elektronik secara otomatis dan terpusat melalui jaringan internet. Sistem pencahayaan konvensional yang masih bersifat manual dinilai kurang efisien, sulit dimonitor secara *real-time*, dan berpotensi menyebabkan pemborosan energi, terutama pada bangunan berskala besar. Oleh karena itu, proyek ini mengembangkan *Centralized Smart Lighting Control System* berbasis IoT sebagai solusi untuk meningkatkan efisiensi energi, kemudahan pengelolaan, serta mendukung konsep *smart building*.

Rumusan Masalah

Berdasarkan latar belakang tersebut, maka rumusan masalah dalam proyek ini adalah :

1. Bagaimana merancang sistem kontrol pencahayaan yang dapat dikendalikan dan dipantau secara terpusat melalui dashboard web?
2. Bagaimana cara mengoptimalkan penggunaan energi listrik melalui otomatisasi lampu menggunakan sensor cahaya?

Tujuan Proyek

Adapun tujuan utama yang ingin dicapai melalui perancangan dan implementasi proyek ini adalah sebagai berikut :

1. Merancang dan mengimplementasikan sistem kontrol lampu berbasis IoT yang dapat dipantau secara *real-time*.
2. Meningkatkan efisiensi energi dengan menerapkan otomatisasi berdasarkan intensitas cahaya lingkungan.
3. Menyediakan sistem pencahayaan yang fleksibel dan mudah dikelola untuk mendukung konsep gedung pintar.

Manfaat Proyek

Kemudian manfaat yang diharapkan dari implementasi sistem ini adalah sebagai berikut :

- Efisiensi Energi : Mampu meminimalisir pemborosan listrik melalui otomatisasi pencahayaan berdasarkan kondisi cahaya lingkungan sekitar.
- Kemudahan Pengelolaan : Memberikan fleksibilitas bagi pengguna untuk mengontrol dan memonitor seluruh lampu secara terpusat dan *real-time*.
- Inovasi Teknologi: Mendukung penerapan konsep *smart building* yang modern melalui integrasi perangkat IoT dan dashboard web.
- Penyimpanan Data Terstruktur: Memungkinkan penyimpanan log aktivitas dan data sensor secara sistematis untuk keperluan analisis lebih lanjut.

BAB II – SPESIFIKASI TEKNIS

Perangkat Lunak (Software Stack)

Pengembangan *Centralized Smart Lighting Control System* berbasis Internet of Things (IoT) menerapkan arsitektur **client-server berbasis web** untuk memungkinkan pengendalian dan pemantauan pencahayaan secara terpusat dan real-time.

1. Frontend (Antarmuka Pengguna)

Frontend sistem dikembangkan menggunakan **React.js** sebagai framework JavaScript untuk membangun antarmuka web yang interaktif dan responsif. Dashboard yang disediakan memungkinkan pengguna untuk :

- Memantau intensitas cahaya lingkungan secara real-time
- Menampilkan status siang dan malam berdasarkan nilai sensor cahaya

React.js dipilih karena kemampuannya dalam pengelolaan *state*, efisiensi performa, serta kemudahan integrasi dengan REST API.

2. Backend (Server & API)

Backend dibangun menggunakan **Node.js** dengan framework **Express.js**. Backend berfungsi sebagai pusat pengolahan data dan pengendalian sistem, dengan tugas utama :

- Menyediakan REST API
- Menerima data intensitas cahaya dari perangkat IoT
- Mengelola logika kontrol lampu (manual dan otomatis)
- Menyimpan data dan log aktivitas ke database

Node.js digunakan karena mendukung komunikasi data secara asinkron dan efisien.

3. Database

Sistem menggunakan **MySQL** sebagai basis data relasional. Database ini menyimpan data secara terstruktur, meliputi :

- Data intensitas cahaya dari sensor LDR
- Data pengguna
- Log aktivitas sistem

MySQL dipilih karena stabilitas dan kemampuannya dalam mengelola relasi data sesuai rancangan ERD.

Perangkat Keras (Hardware & IoT Layer)

Lapisan perangkat keras berfungsi sebagai pengambil data lingkungan dan pelaksana perintah pengendalian pencahayaan.

4. Mikrokontroler

Sistem menggunakan **ESP32 atau ESP8266** sebagai mikrokontroler utama. Perangkat ini dipilih karena :

- Memiliki modul Wi-Fi terintegrasi
- Mendukung komunikasi data berbasis internet
- Cocok untuk implementasi sistem IoT skala kecil hingga menengah

Mikrokontroler bertugas mengolah data sensor dan mengirimkannya ke server pusat.

5. Sensor Cahaya (LDR – Light Dependent Resistor)

Sistem menggunakan **sensor cahaya LDR** untuk mendeteksi tingkat intensitas cahaya lingkungan. Nilai resistansi LDR akan berubah sesuai dengan kondisi pencahayaan sekitar, sehingga dapat digunakan untuk menentukan :

- Kondisi siang atau malam
- Keputusan otomatis menyalakan atau mematikan lampu

Data intensitas cahaya yang diperoleh dari LDR dikirimkan secara berkala ke server backend untuk diproses dan disimpan.

6. Perangkat Pencahayaan (Aktuator)

Lampu berfungsi sebagai aktuator yang dikendalikan melalui **relay** yang terhubung ke mikrokontroler. Lampu dapat dioperasikan berdasarkan :

- Perintah manual dari dashboard web
- Logika otomatis sistem berdasarkan ambang batas (threshold) nilai LDR

Seluruh perangkat IoT terhubung ke jaringan internet sehingga memungkinkan kontrol dan monitoring secara terpusat.

Protokol Komunikasi dan Keamanan

7. Protokol Komunikasi

Komunikasi antar perangkat IoT, backend, dan frontend dilakukan menggunakan :

- **HTTP/HTTPS**
- **REST API**

Alur komunikasi sistem adalah sebagai berikut:

1. Sensor LDR membaca intensitas cahaya lingkungan

2. Mikrokontroler mengirimkan data ke backend melalui REST API
3. Backend memproses dan menyimpan data ke database MySQL
4. Frontend menampilkan data secara real-time melalui dashboard web

Pendekatan ini memungkinkan sistem berjalan secara terpusat dan efisien.

8. Keamanan Sistem

Keamanan sistem diterapkan dengan :

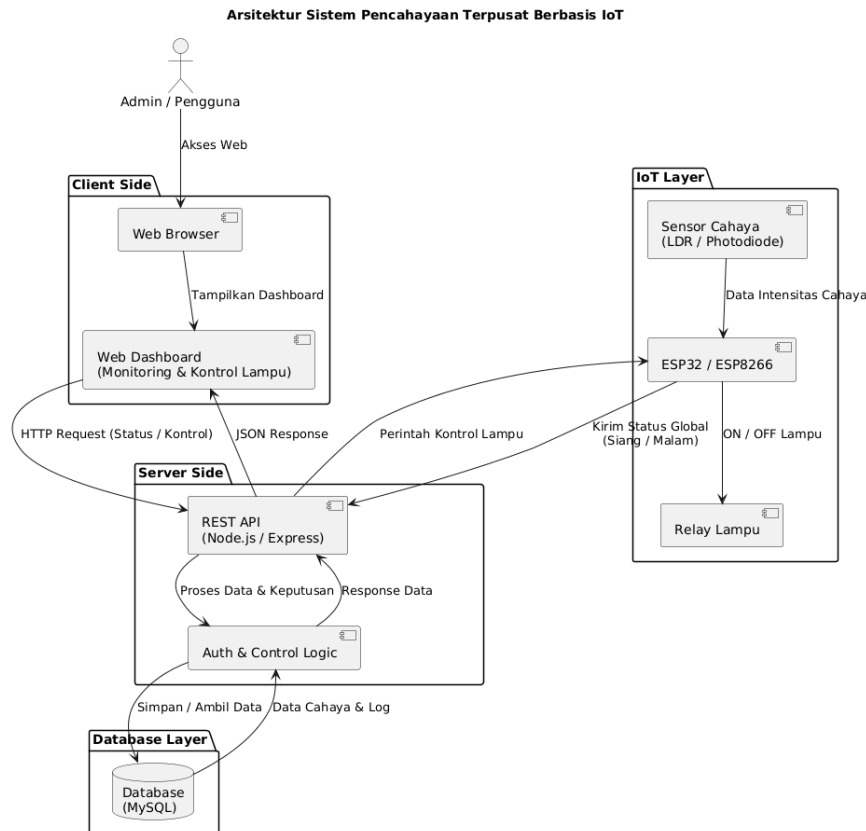
- Pembatasan akses API melalui endpoint tertentu
- Pengelolaan hak akses pengguna
- Penyimpanan data secara terstruktur di database

Pada tahap pengembangan lanjutan, keamanan dapat ditingkatkan melalui :

- Autentikasi berbasis token
- Enkripsi data menggunakan HTTPS
- Manajemen sesi pengguna

BAB III– PERANCANGAN SISTEM (UML)

Perancangan ini menggunakan standar *Unified Modeling Language* (UML) untuk memvisualisasikan struktur dan perilaku sistem secara komprehensif.

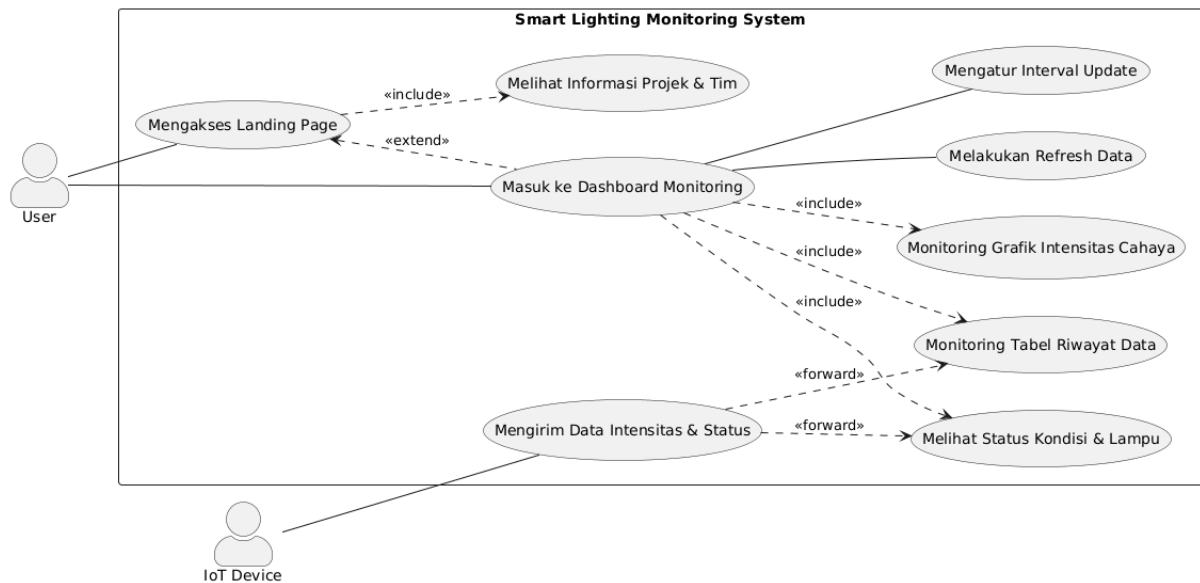


3.1 Arsitektur Sistem

Sistem dirancang menggunakan pendekatan *Layered Architecture* untuk memastikan skalabilitas dan kemudahan pemeliharaan :

- **Client Side (Frontend Layer)** : Berjalan pada Web Browser pengguna. Lapisan ini bertanggung jawab menampilkan dashboard interaktif untuk monitoring intensitas cahaya dan menyediakan tombol kontrol lampu.
- **Server Side (Application Layer)** : Bertindak sebagai otak sistem yang dibangun dengan Node.js dan Express. Mengelola logika autentikasi, memproses permintaan HTTP dari pengguna, dan mengirimkan perintah ke perangkat IoT.
- **IoT Layer (Physical Layer)** : Terdiri dari mikrokontroler (ESP32/ESP8266) yang terhubung ke sensor cahaya dan relay lampu. Lapisan ini melakukan pembacaan fisik lingkungan dan eksekusi perangkat keras.
- **Database Layer (Data Layer)** : Menggunakan MySQL untuk menyimpan data persisten seperti log aktivitas pengguna, data historis sensor cahaya, dan informasi akun pengguna.

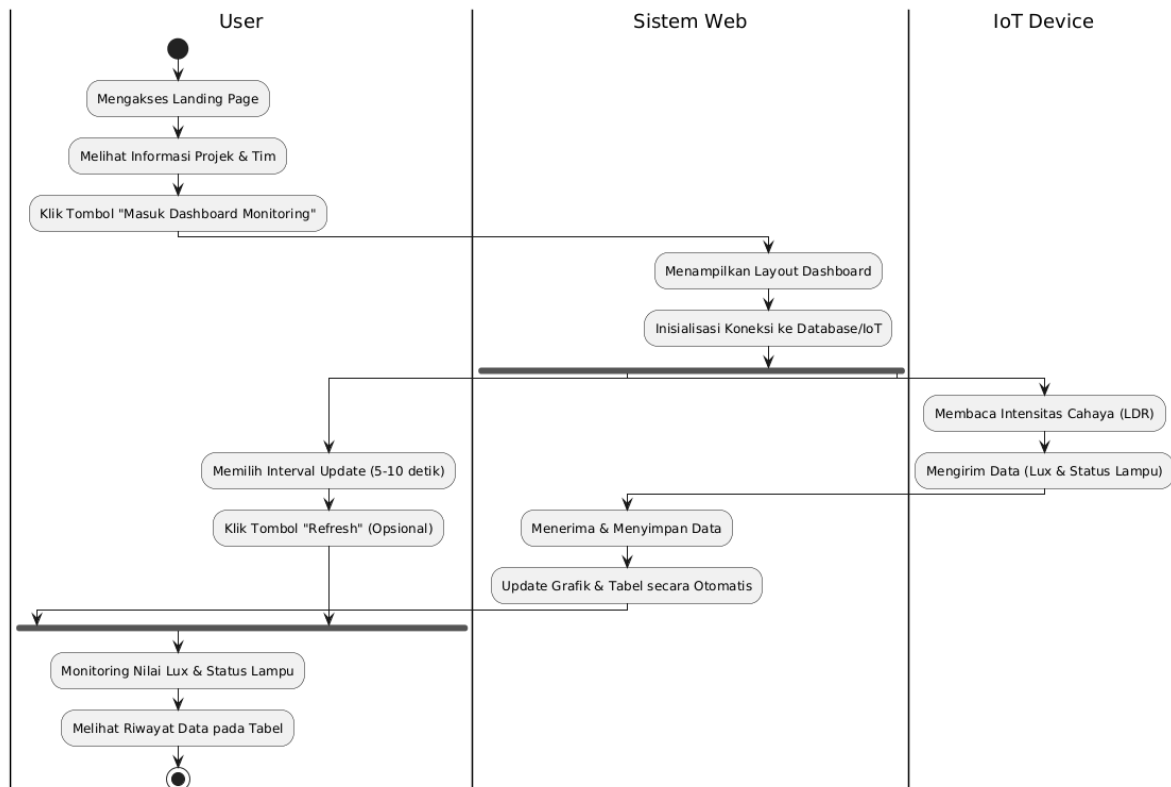
3.2 Use Case Diagram



Sistem mendefinisikan tiga aktor utama dengan hak akses yang berbeda dalam lingkup *Centralized Smart Lighting Control System* :

- **Admin** : Memiliki kontrol penuh terhadap sistem. Hak akses meliputi melihat riwayat aktivitas, kontrol lampu manual, mengubah mode operasi (otomatis/manual), serta manajemen sesi login/logout.
- **User** : Peran pengguna biasa yang memiliki akses terbatas. Hanya diizinkan untuk melakukan monitoring intensitas cahaya secara real-time dan melihat status siang/malam.
- **IoT Device** : Aktor sistem non-manusia yang secara otomatis mengirimkan data sensor cahaya ke server agar dapat diproses lebih lanjut.

3.3 Alur Kerja (Activity Diagram)

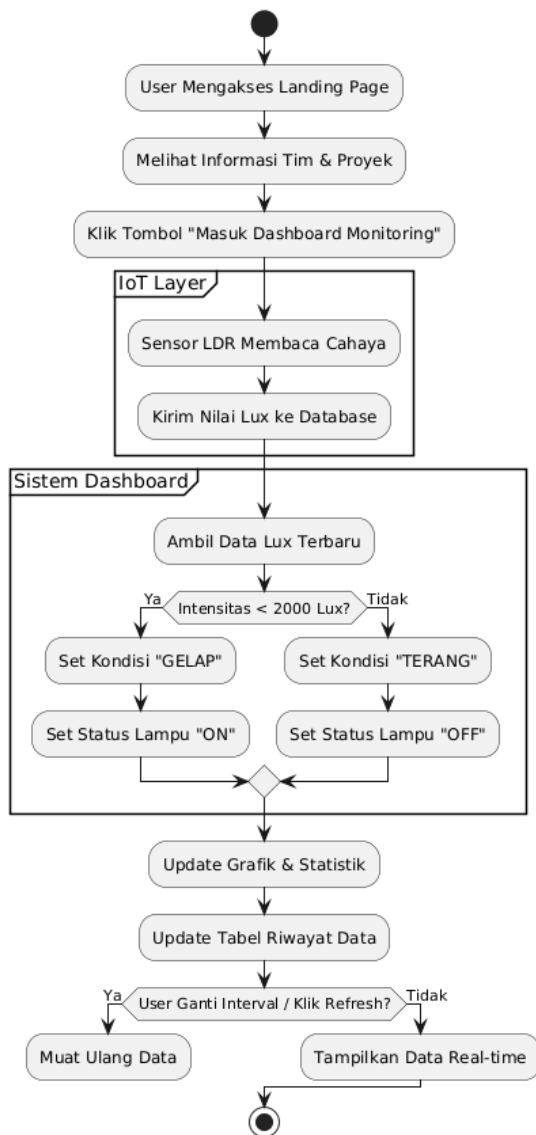


Alur operasional sistem dijelaskan melalui rangkaian aktivitas sebagai berikut :

1. **Validasi Akses** : Pengguna memulai dengan membuka web dashboard. Sistem melakukan pengecekan status login; jika belum, pengguna diarahkan ke halaman login untuk input kredensial dan validasi.
2. **Pemrosesan Data** : Setelah login berhasil, sistem secara otomatis mengambil data terbaru dari sensor cahaya dan menampilkannya sebagai status siang atau malam pada dashboard.
3. **Logika Pengambilan Keputusan** : Sistem mengecek mode yang sedang aktif:
 - a. **Mode Manual** : User memiliki otoritas penuh untuk menekan tombol ON/OFF lampu melalui dashboard.
 - b. **Mode Otomatis** : Sistem secara mandiri memberikan instruksi kepada relay berdasarkan ambang batas (threshold) cahaya yang diterima sensor.
4. **Audit Trail & Sesi** : Setiap perubahan status atau perintah yang diberikan akan disimpan ke dalam log aktivitas sebagai rekam jejak. Sesi berakhir ketika pengguna memilih untuk logout.

3.4 Flowchart Sistem

Flowchart Sistem Monitoring - Project Vanguard 5



Flowchart ini menggambarkan siklus hidup data dalam aplikasi **Project Vanguard 5**, yang terbagi menjadi tiga fase utama:

1. Fase Inisialisasi & Navigasi (User Interface)

- **Akses Landing Page:** Proses dimulai saat User membuka alamat URL aplikasi. Sistem akan menampilkan halaman depan yang berisi informasi tim dan deskripsi proyek.
- **Interaksi User:** User harus melakukan aksi klik pada tombol "**Masuk Dashboard Monitoring**". Ini adalah pintu gerbang (gate) sebelum sistem mulai menjalankan fungsi monitoring secara aktif.

2. Fase Akuisisi Data (IoT Layer)

- **Pembacaan Sensor:** Setelah dashboard terbuka, sistem memerintahkan perangkat IoT (ESP32/Arduino) untuk mengaktifkan sensor LDR.
- **Pengiriman Data:** Nilai intensitas cahaya (Lux) yang ditangkap oleh sensor dikirimkan ke server/database melalui protokol komunikasi (HTTP/MQTT).

3. Fase Logika Keputusan (Processing Layer)

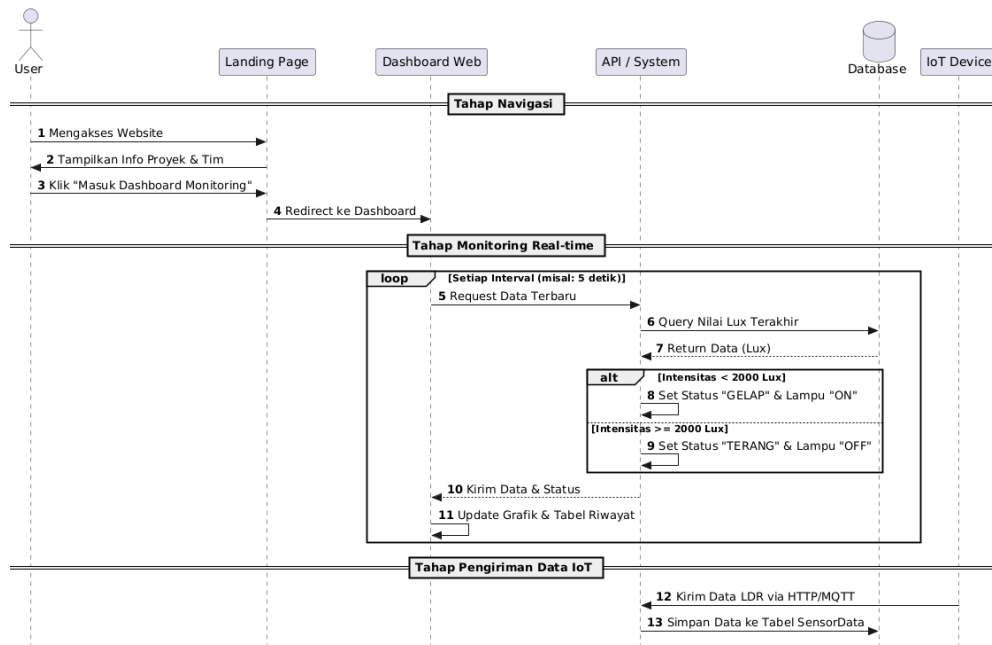
Di tahap ini, sistem melakukan pengecekan berdasarkan **Ambang Batas (Threshold)** yang telah ditentukan sebesar **2000 Lux**:

- **Kondisi Gelap:** Jika nilai Lux < **2000**, maka sistem secara otomatis menetapkan status kondisi menjadi **"GELAP"** dan memberikan perintah pada lampu untuk **"ON"**.
- **Kondisi Terang:** Jika nilai Lux $\geq$ **2000**, maka sistem menetapkan status kondisi menjadi **"TERANG"** dan mematikan lampu (**"OFF"**).

4. Fase Output & Pembaruan (Monitoring Layer)

- **Visualisasi:** Hasil pengolahan data ditampilkan ke dalam dua format di Dashboard:
 - **Grafik:** Untuk melihat tren perubahan cahaya.
 - **Tabel Riwayat:** Untuk mencatat detail record setiap detiknya (ID, Lux, Kondisi, Status Lampu).
- **Looping (Refresh):** Sistem memiliki mekanisme pengecekan interval (misal: 5 detik). Jika waktu tunggu tercapai atau user menekan tombol "Refresh", sistem akan kembali ke tahap pengambilan data terbaru agar informasi tetap *real-time*

3.5 Sequence Diagram Monitoring



Sequence Diagram ini menggambarkan urutan interaksi antar komponen (User, Website, Server, Database, dan Hardware) dalam satu garis waktu. Berikut adalah rincian prosesnya:

1. Navigasi Awal (Langkah 1-4)

Proses dimulai ketika **User** mengakses **Landing Page**. Saat tombol "Masuk Dashboard" diklik, terjadi perpindahan kontrol dari halaman informasi ke **Dashboard Web**. Di sinilah sistem mulai menyiapkan koneksi untuk pemantauan data.

2. Mekanisme Data Request (Looping)

Sesuai dengan fitur "Waktu Update" yang ada di web Anda (misalnya 5 detik), sistem menjalankan **Looping**.

- **Request:** Dashboard mengirimkan instruksi ke **Backend API** untuk mengambil data terbaru.
- **Query:** Server melakukan permintaan ke **Database** untuk mengambil record terakhir dari tabel SensorLogs (kolom cahaya).

3. Proses Logika Keputusan (Alt Fragment)

Ini adalah bagian terpenting yang menjelaskan kecerdasan sistem Anda. Terjadi pengecekan kondisi pada sisi Server:

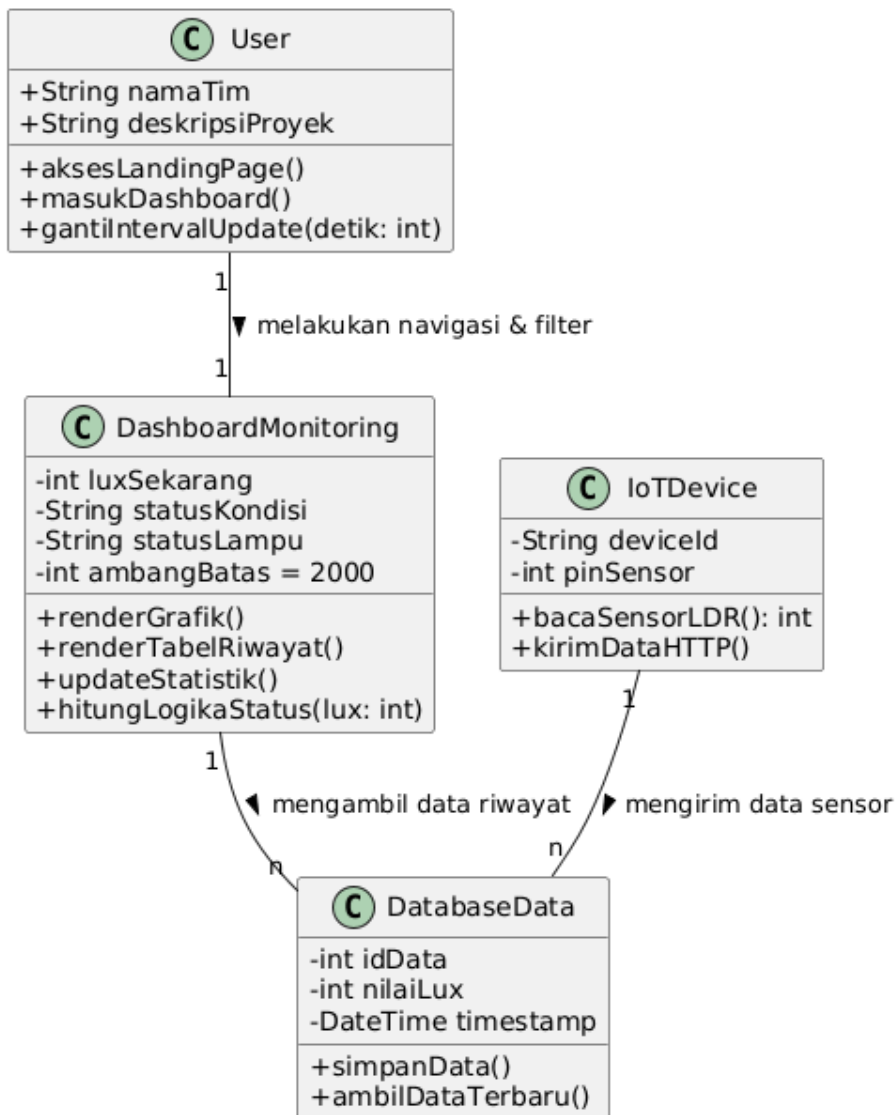
- **Jika Cahaya < 2000 Lux:** Server menentukan status sebagai "**GELAP**" dan lampu "**ON**".
- **Jika Cahaya >= 2000 Lux:** Server menentukan status sebagai "**TERANG**" dan lampu "**OFF**". Hasil keputusan ini kemudian dikirimkan kembali ke Dashboard untuk mengubah tampilan UI secara dinamis.

4. Sinkronisasi Hardware (IoT Layer)

Secara paralel dan terus-menerus, **IoT Device (ESP32/LDR)** mengirimkan data cahaya yang baru saja dibaca ke Server. Server kemudian menyimpannya ke Database. Proses ini memastikan bahwa setiap kali Dashboard melakukan *request*, data yang tersedia di Database adalah data yang paling baru (*up-to-date*).

3.6 Class Diagram

Class Diagram - Project Vanguard 5



1. Komponen Frontend (Client-Side)

Ini adalah representasi dari kode React.js yang Anda gunakan untuk tampilan:

- **SensorDashboard:** Merupakan "Parent Class" atau komponen utama. Ia memiliki *state* untuk menyimpan data sensor dan fungsi `useEffect()` untuk mengambil data dari API secara berkala.
- **SensorChart & SensorHistory:** Adalah kelas spesialisasi. `SensorChart` fokus pada pengolahan data menjadi grafik, sementara `SensorHistory` fokus pada pemetaan data ke dalam bentuk tabel riwayat.
- **Hubungan:** Dashboard memiliki hubungan **Composition** (tanda diamond hitam) terhadap Chart dan History, artinya jika Dashboard ditutup, maka komponen grafik dan tabel juga ikut hilang.

2. Komponen Backend (Server-Side)

Ini adalah logika Node.js/Express yang mengelola data:

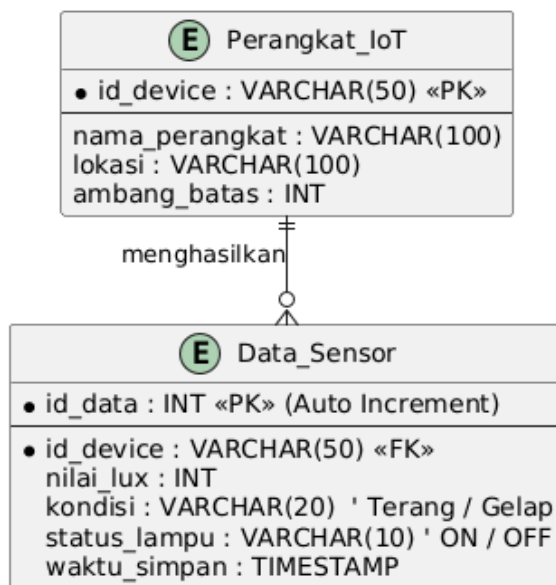
- **IotController:** Bertugas sebagai pengatur lalu lintas. Ia menerima permintaan dari web, lalu memerintahkan model untuk mengambil data dari database.
- **IotRoutes:** Berfungsi sebagai gerbang (endpoint). Ia memetakan alamat URL (seperti /api/logs) ke fungsi yang ada di Controller.

3. Komponen Data (Model & Database)

- **SensorLogModel:** Merupakan representasi dari tabel SensorLogs yang Anda buat di file migrasi. Ia mendefinisikan atribut seperti idSensorLogs dan cahaya.
- **DatabaseData:** Kelas yang menangani koneksi fisik ke MySQL/PostgreSQL menggunakan library Sequelize.

3.6 ERD

Entity Relationship Diagram (ERD) - Project Vanguard 5



ERD menggambarkan struktur penyimpanan data permanen dalam sistem Anda. Berdasarkan kode migrasi yang Anda miliki, fokus utama adalah entitas **SensorLogs**.

1. Komponen Entitas dan Atribut

- **Entitas SensorLogs:** Merupakan objek utama yang menyimpan setiap kejadian (log) yang dikirim oleh sensor LDR.
- **idSensorLogs (Primary Key):** Atribut kunci yang unik. Fungsinya untuk memastikan setiap baris data di tabel database memiliki identitas berbeda dan tidak tertukar.

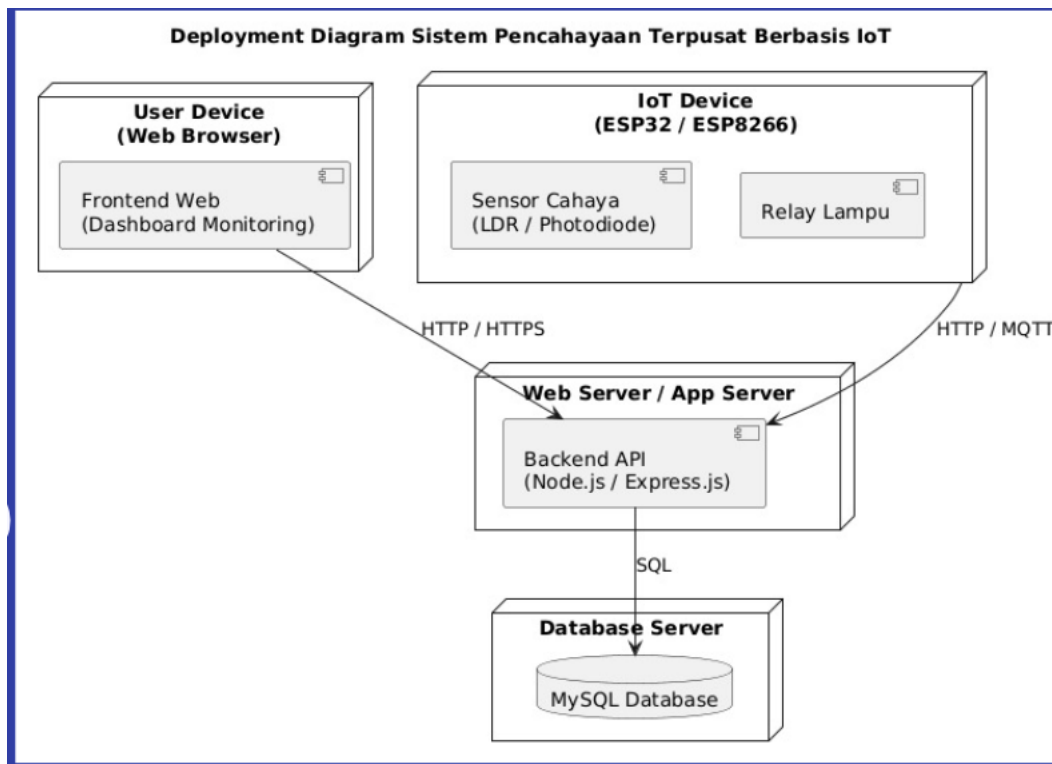
- **cahaya (Integer):** Atribut yang menyimpan nilai numerik intensitas cahaya. Data inilah yang akan diolah menjadi grafik di halaman dashboard.
- **createdAt & updatedAt (Timestamp):** Atribut otomatis yang mencatat kapan data sensor masuk ke sistem. Kolom ini sangat penting untuk menampilkan "TANGGAL & WAKTU" pada tabel riwayat di web.

2. Hubungan (Relationship)

Dalam sistem IoT sederhana, relasi yang terjadi adalah antara **Perangkat Hardware** dan **Tabel Log**:

- **One-to-Many (1:N):** Satu perangkat IoT (ESP32) dapat menghasilkan banyak baris data sensor (SensorLogs) seiring berjalannya waktu. Itulah sebabnya tabel riwayat Anda bisa terus bertambah setiap 5 detik.

3.7 Deployment System



A. Client Side (Hardware)

- **ESP32 & LDR:** Perangkat keras dideploy di lokasi pemantauan. Perangkat ini dikonfigurasi dengan *SSID* dan *Password* WiFi agar bisa terhubung ke internet. Kode program (C++) di-burn ke dalam mikrokontroler untuk melakukan pengiriman data secara terus-menerus.

B. Network (Internet)

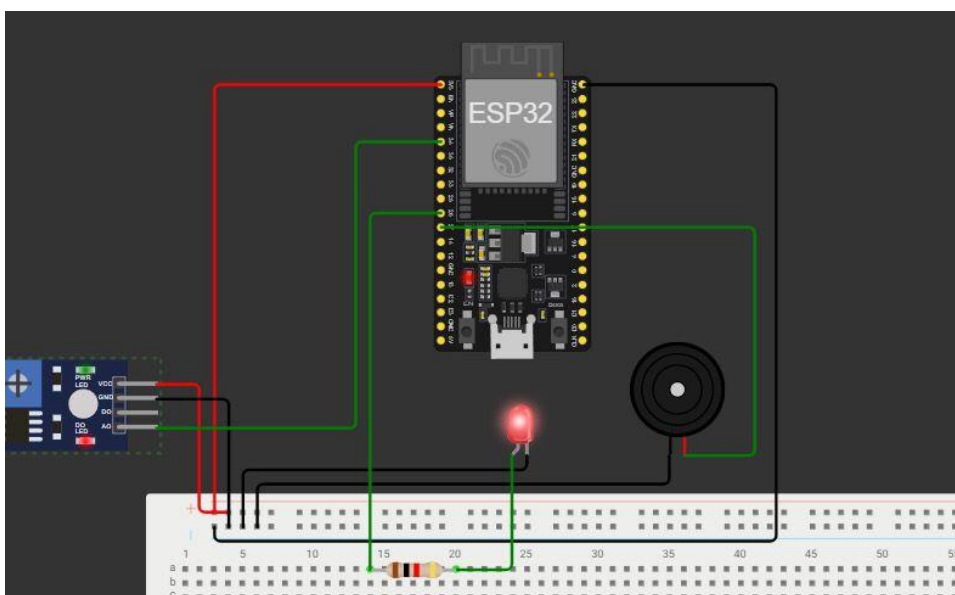
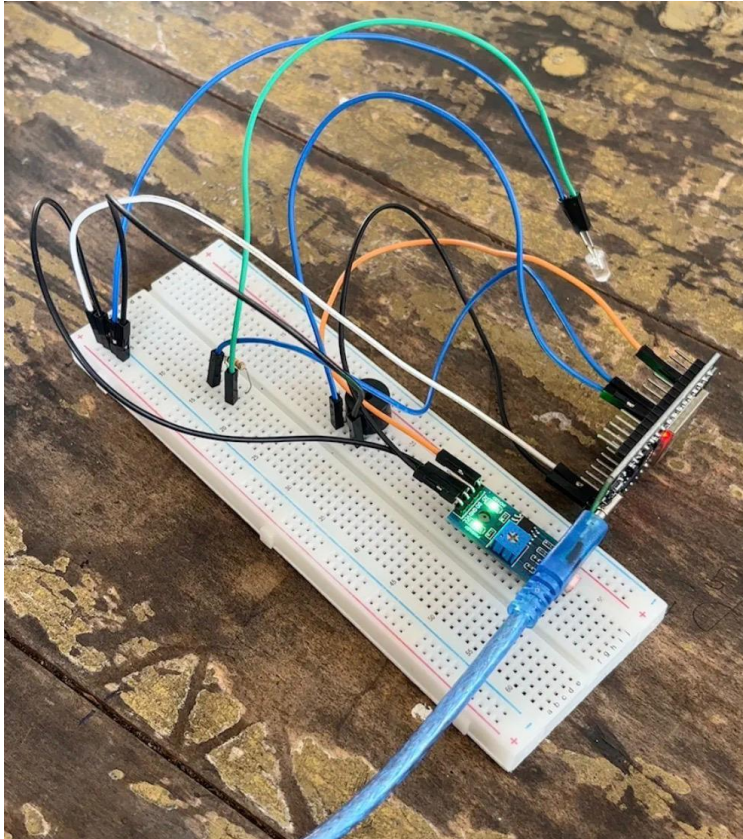
- Bertindak sebagai jembatan. Data dari sensor dikirimkan melalui protokol HTTP (atau MQTT jika Anda menggunakannya). Sistem deployment ini memastikan bahwa IP Address server dapat dijangkau oleh perangkat IoT.

C. Server Side (Backend & Database)

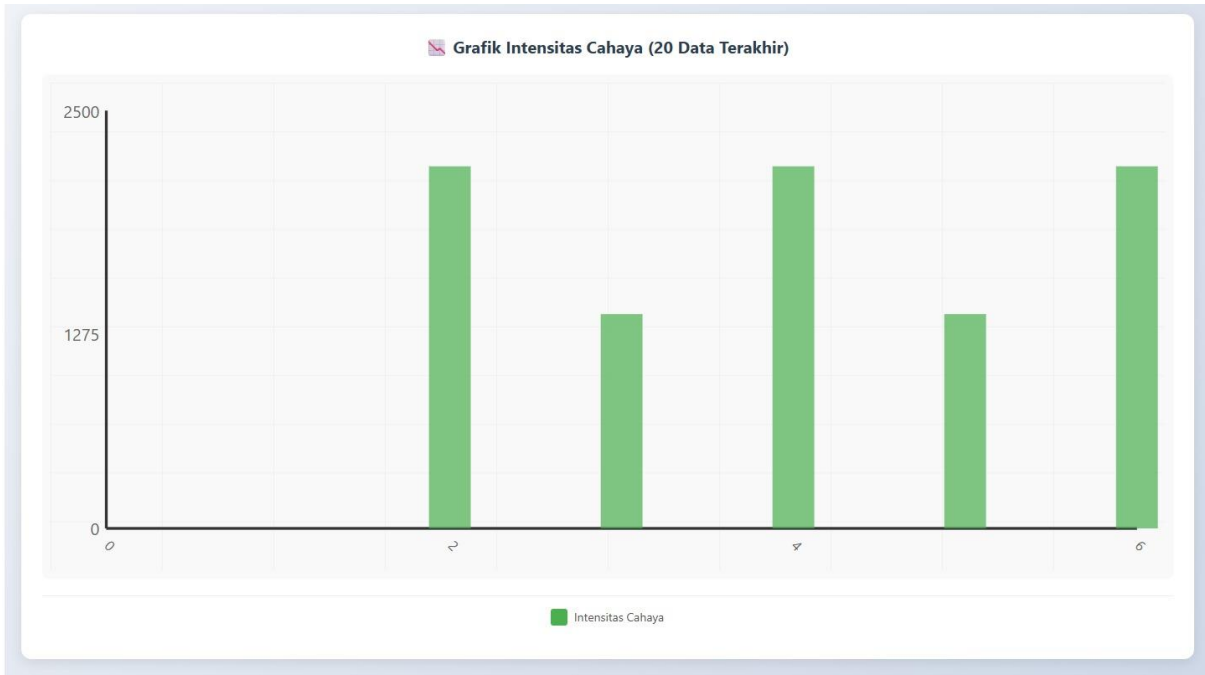
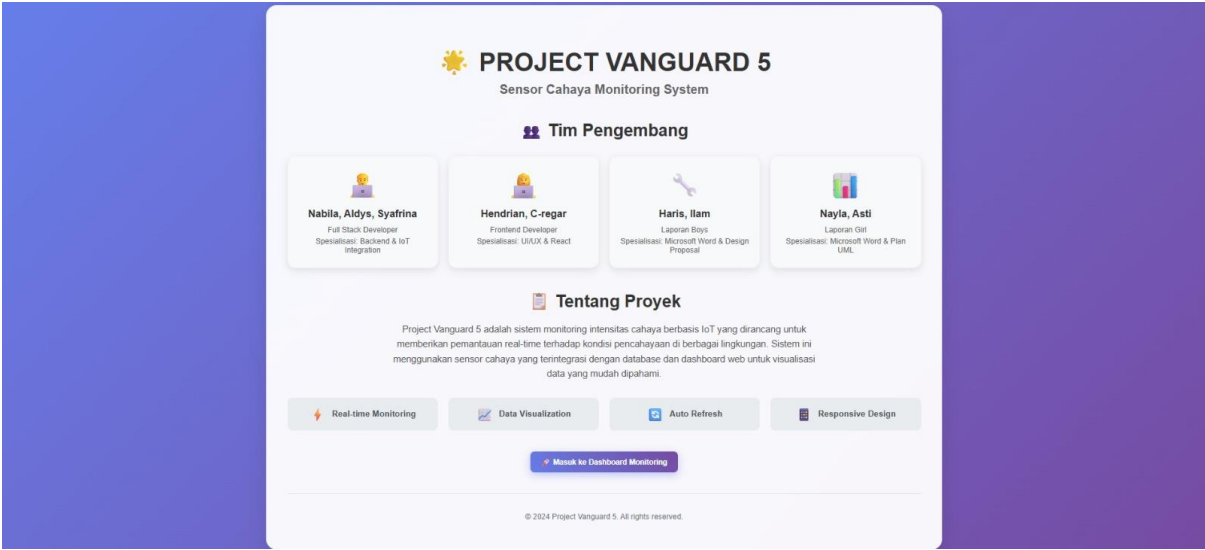
- Web Server (Node.js): Tempat di mana kode *Project Vanguard 5* dijalankan. Server ini menangani permintaan (request) dari dua arah: dari alat IoT untuk simpan data, dan dari browser user untuk tampilkan data.
- Database (MySQL): Dideploy sebagai tempat penyimpanan permanen. Struktur tabel dibuat menggunakan file Migration Sequelize yang sebelumnya Anda buat.

BAB IV– APLIKASI

4.1 Dokumentasi Alat



4.2 Tampilan Web



Riwayat Data Sensor

NO	ID SENSOR	INTENSITAS CAHAYA	KONDISI	STATUS LAMPU	TANGGAL & WAKTU
1	7	2500 lux	TERANG	OFF	13/1/2026, 19.38.31
2	6	1500 lux	GELAP	ON	13/1/2026, 19.38.09
3	5	2500 lux	TERANG	OFF	13/1/2026, 19.38.01
4	4	1500 lux	GELAP	ON	13/1/2026, 19.32.43
5	3	2500 lux	TERANG	OFF	13/1/2026, 19.32.36
6	2	50 lux	TERANG	ON	13/1/2026, 17.12.50
7	1	50 lux	TERANG	ON	13/1/2026, 17.06.58

Sebelumnya

Halaman 1 dari 1 | Total: 7 data

Selanjutnya

Statistik Data Sensor

NILAI TERTINGGI

2500 lux

NILAI TERENDAH

50 lux

RATA-RATA

1514.29 lux

TOTAL PENGUKURAN

7

4.3 Susunan Folder dan Source Code

BAB V– KESIMPULAN

Berdasarkan hasil perancangan dan implementasi proyek **Centralized Smart Lighting Control System**, dapat ditarik beberapa poin kesimpulan utama sebagai berikut :

- **Efektivitas Monitoring Lingkungan** : Sistem ini berhasil mempermudah pengawasan kondisi cahaya lingkungan secara terpusat dan *real-time* melalui *dashboard web* yang interaktif. Pengguna tidak lagi perlu melakukan pengecekan fisik, melainkan cukup memantau visualisasi data yang disajikan secara akurat.
- **Efisiensi Pengelolaan Energi** : Melalui penerapan otomatisasi pada *mode* otomatis, sistem mampu mengoptimalkan penggunaan energi listrik dengan menyesuaikan status lampu berdasarkan intensitas cahaya aktual yang ditangkap oleh sensor LDR. Hal ini menjadi solusi konkret dalam mengurangi pemborosan energi pada bangunan berskala besar.
- **Integritas dan Struktur Data** : Seluruh interaksi pengguna, mulai dari *login* hingga kontrol lampu, serta data historis dari sensor, tersimpan secara sistematis dan terstruktur di dalam database MySQL. Dokumentasi log aktivitas ini sangat penting untuk keperluan audit penggunaan energi dan pemantauan keamanan sistem.
- **Keandalan Arsitektur Sistem** : Penggunaan arsitektur berbasis REST API dengan *stack* React.js, Node.js, dan ESP32 memberikan fondasi yang solid dan stabil. Pemisahan antara *client-side*, *server-side*, dan *IoT layer* memungkinkan sistem bekerja secara modular dan meminimalkan risiko kegagalan sistem secara menyeluruh.
- **Skalabilitas dan Potensi Masa Depan** : Sistem ini dirancang dengan fleksibilitas tinggi, sehingga memiliki potensi besar untuk dikembangkan lebih lanjut. Di masa depan, arsitektur ini memungkinkan integrasi perangkat IoT yang lebih kompleks, seperti sensor gerak (PIR), sensor suhu (DHT), atau penggunaan protokol komunikasi yang lebih ringan seperti MQTT untuk efisiensi data yang lebih baik.